

# Week 05 Explained Student Worksheet

## *Boolean Retrieval Using Binary Vectors*

Name \_\_\_\_\_ Student ID \_\_\_\_\_ Group \_\_\_\_\_

Evaluate AND and NOT queries over the Week 4 incidence matrix. Trace the vectors before interpreting the matched document IDs.

Use this worksheet during the two-hour practical session. Read, predict, run, and explain. The existing lab sheet contains the complete program; this companion explains key ideas and provides space to record your own evidence. Python 3.10 or later is sufficient; no pip packages are needed.

### What you should be able to do

- Translate query terms into binary vectors.
- Apply AND and NOT operations over a fixed document universe.
- Implement query processing without external search libraries.
- Compare hand predictions with intermediate vectors and returned IDs.

### Understand the concepts

Boolean retrieval returns documents that satisfy a logical condition. AND requires all positive conditions. NOT complements a condition relative to the loaded collection. For 0/1 values, a & b performs conjunction, and 1 - bit performs complement. Python ~bit is unsuitable here because it returns negative integers rather than a 0/1 complement.

For the original collection, python is [1, 0, 0, 1, 0] and java is [0, 1, 0, 0, 1]. NOT java is [1, 0, 1, 1, 0], so python AND NOT java is [1, 0, 0, 1, 0]. The final 1 positions identify doc1 and doc4.

The supported grammar is [NOT] term (AND [NOT] term)\*. Operators are case-insensitive and must be separate words. Each term contains letters only. OR, parentheses, and implicit conjunction are outside this lab. A positive unknown term gives an all-zero vector; NOT unknown gives all ones.

The original scripts split on the substring AND, which can split words such as pandas. This version splits on regex word boundaries and validates each clause. Query terms are lowercased and looked up directly; stop words have no indexed row. A stop word therefore behaves as an unknown term in this Boolean grammar.

### Predict before running

For query python AND NOT java, state the set of documents against which NOT must be evaluated.

---

## Prepare your terminal

Extract the course and open a terminal inside Week\_05\_Boolean\_Retrieval. Keep docs and data beside the Python files. The main demonstration is lab05\_solution.py; your exercise starter is lab05\_exercises.py. Starters run but deliberately contain unfinished TODO functions.

```
py -3.13 lab05_solution.py
py -3.13 lab05_exercises.py
```

Use py -3 or python instead if that is your working Python 3.10+ command.

## Worked example with explanation

Predict all three vectors in this complete example, then execute it. The term vectors match the shipped collection.

```
python_bits = [1, 0, 0, 1, 0]
java_bits = [0, 1, 0, 0, 1]
not_java = [1 - bit for bit in java_bits]
result = [python_bits[i] & not_java[i] for i in range(5)]
print(python_bits)
print(not_java)
print(result)
```

### Expected output from an executed example

```
[1, 0, 0, 1, 0]
[1, 0, 1, 1, 0]
[1, 0, 0, 1, 0]
```

### How each stage produces the result

1. Each vector has five coordinates in order doc1 through doc5. The python row marks its two matching documents.
2. The list comprehension `1 - bit` swaps 0 and 1. NOT java therefore includes every loaded document that does not contain java.
3. The `&` operation on two binary integers returns 1 only when both inputs are 1. Applying it at each position enforces both query conditions.
4. Map final 1 positions back to document IDs. The baseline grammar supports [NOT] term followed by AND clauses; OR and parentheses are not part of this program.

Explain one intermediate value in your own words

---



---

## Read the weekly program

The following excerpt is copied exactly from lab05\_solution.py. It depends on the imports, constants, and helper functions in that file. Read it there rather than pasting it into an empty script and expecting the excerpt to run alone.

```
def run_query(query, matrix, doc_names, trace=True):
    result = None
    for negative, term in parse_query(query):
        bits = matrix.get(term, [0] * len(doc_names))
        if trace:
            print(term, "->", bits)
        if negative:
            bits = [1 - bit for bit in bits]
            if trace:
                print("NOT", term, "->", bits)
        if result is None:
            result = bits[:]
        else:
            result = [result[i] & bits[i] for i in range(len(bits))]
    matched = [doc_names[i] for i, bit in enumerate(result) if bit == 1]
    if trace:
        print("Result vector:", result)
    return matched
```

### Code walkthrough

1. parse\_query returns (negative, term) pairs. A positive unknown term receives an all-zero row from matrix.get.
2. A negative clause complements that row over the entire doc\_names universe. NOT unknown therefore produces all ones in this baseline.
3. result is initially None so the first clause initializes it. Later clauses combine with the accumulated result using coordinate-wise AND.
4. The final comprehension translates bits into IDs. Word-boundary parsing prevents the letters and inside pandas from being mistaken for an operator.

Predict python AND unknown and python AND NOT unknown. Explain why their results differ.

---



---

### Connection to the course

Week 6 keeps the same Boolean meaning but represents matches as sorted posting lists. Compare both methods on the same collection before comparing their outputs.

## Guided practical activity

Work through the steps in order. Compare your predictions with observations and record at least one specific value per experiment. Keep original baseline data until you have saved the first transcript. Put experiments that change files in a disposable copy.

**Step 1** Reuse the exact Week 4 docs collection and matrix builder.

Keep the Week 4 matrix and its document-order labels together; a vector alone is not enough to interpret matches.

**Step 2** Before running, predict results for python AND data, machine AND learning, python AND NOT java, and data AND analysis.

Record your predicted IDs and result bits before opening the terminal demonstration.

**Step 3** Run the default demonstration and compare every intermediate vector with your predictions.

Inspect the trace for each clause. An intermediate complement can reveal an error hidden by a final empty answer.

**Step 4** Trace `parse_query` and explain the tuple (negative, term) returned for each clause.

Explain the Boolean flag and term for each parsed clause. Invalid syntax is rejected rather than guessed.

**Step 5** Complete `and_not` in the starter. Then add a function mapping 1 positions to IDs.

Implement `and_not` in the starter yourself. Check its vectors have equal length and binary values.

**Step 6** Run `python lab05_solution.py --interactive`. Test an unknown term, a negated unknown term, and invalid syntax. Type quit to exit.

Test unknown positive and negated terms separately. Also test OR and record the error message; quit ends the interactive loop.

### Test record

| Check to demonstrate                            | Actual observation | Pass or revise |
|-------------------------------------------------|--------------------|----------------|
| The four default queries match hand predictions |                    |                |
| An unknown positive term produces no matches    |                    |                |
| Unsupported OR syntax is rejected               |                    |                |

Record one prediction that changed after running the program and explain why

---



---

## Independent work and submission

Complete these selected tasks in your own exercise file. Use the existing lab sheet for additional exercises. Do not submit the unchanged starter or simply import a solution in place of implementing the requested function.

- Prediction: manually evaluate the four default queries and record both bit vectors and IDs.
- Implementation: complete `and_not` without importing the solution.
- Debugging: replace 1 - bit with `~bit` in a disposable copy. Explain the negative values produced.
- Interpretation: predict python AND unknown and python AND NOT unknown before running.

### Diagnose common errors

**NOT produces negative integers.** The Python bitwise operation `~0` gives -1 and `~1` gives -2. Complement binary presence using 1 - bit.

**A word containing and is split.** Split on the operator word with boundaries, not on its characters wherever they occur inside another term.

### Submit these items

- Your completed starter and manual predictions for all four default queries.
- A trace of python AND NOT java, including the original and complemented java vectors.
- Three edge-case queries with the expected and observed behavior.

Your evidence should identify the input or query, expected result, actual result, and explanation. State any document edits or preprocessing changes so the result can be reproduced. Include this completed worksheet or a clearly numbered digital response.

### Exit questions

What condition does AND enforce?

---

Relative to which set does NOT operate?

---

### Optional challenge

Support a parenthesized expression language only after defining its grammar and precedence. Keep the simple baseline available and compare results on expressions supported by both versions.

### Source for this companion

Based on the supplied Week 05 lab sheet, shipped program, exercise starter, and sample data. The complete main program and `expected_output.txt` in this folder are the references for checking your baseline run.